Universidade do Minho
Escola de Engenharia

Engenharia Informática
Sistemas Operativos

Sistema de Gestão de ficheiros

Nome/Número
Miguel Santos a72443

Data
30 / 04 /2025

Índice

1 Introdução **3**
1.1 Introdução . . . 3

2 Arquitetura **4**
2.1 Possíveis metodologias . . . 4
2.1.1 Primeira arquitetura . . . 4
2.1.2 Segunda arquitetura . . . 5
2.2 Comunicação entre Cliente e Servidor . . . 5
2.3 Modulo Cliente . . . 6
2.3.1 Formatação do Pedido . . . 6
2.3.2 Criação e uso de pipes no módulo Cliente . . . 7
2.4 Modulo Mem . . . 7
2.4.1 DocInfo e chavePath . . . 7
2.4.2 Organização de DocInfo e ChavePath . . . 7
2.4.3 Funcionalidades Principais . . . 8
Inserção . . . 8
Consulta . . . 8
Remoção . . . 8
Gestão da Cache . . . 9
Estrutura em uso na query -s . . . 9
2.5 Modulo Servidor . . . 9
2.5.1 Criação do Pipe Cliente->Servidor e leitura do mesmo . . . 10
2.5.2 Processar Pedidos . . . 10
Adicionar,remover e consulta . . . 10
Operações de Procura . . . 10
Operação -l . . . 11
Operação -s . . . 11
Servidor não bloqueante . . . 12
Gestão processos zombie . . . 12
Funções extra adicionais . . . 13
Gestão de duplicados . . . 13
Método de gerar keys . . . 13
Cuidados com a memoria . . . 13

3 Resultados e Analise **14**
3.0.1 Testes iniciais a cada tipo de cache . . . 14
3.0.2 Testes relativos a duplicação . . . 14

4 Conclusão **16**

Capítulo 1

Introdução

1.1 Introdução

Neste trabalho foi pedido para desenvolver e implementar um sistema de gestão de ficheiros. Este sistema tem com base algumas funcionalidades típicas num sistema de gestão de ficheiros como:

- Indexação da meta informação do ficheiro (key,titulo,autores,ano,path)
- Consulta da meta informação
- remoção da meta informação

Este sistema irá contar também com duas funcionalidades extra sendo elas:

- Pesquisa do numero de ocorrências de uma dada palavra chave num dado ficheiro
- Pesquisa uma dada palavra chave no sistema e devolve uma lista com os ficheiros que tem essa mesma palavra

Este trabalho irá ser implementado em linguagem C e irá explorar conceitos de processos e gestão dos mesmos, comunicação entre processos, chamadas ao sistema e persistência de dados. Este trabalho irá se dividir em 2 grandes módulos, o cliente e o servidor. O cliente irá apenas fazer um pedido ao servidor, esperar a resposta do servidor e imprimir a mesma para o std output. O servidor vai ser responsável por toda a carga do trabalho sendo este responsável pela gestão da base de dados e processamento de pedidos sendo que o processamento e divido em 3 etapas sendo estas: receção do pedido, processamento e envio de resposta para o cliente. Sendo assim conseguimos identificar 3 pontos críticos neste projeto:

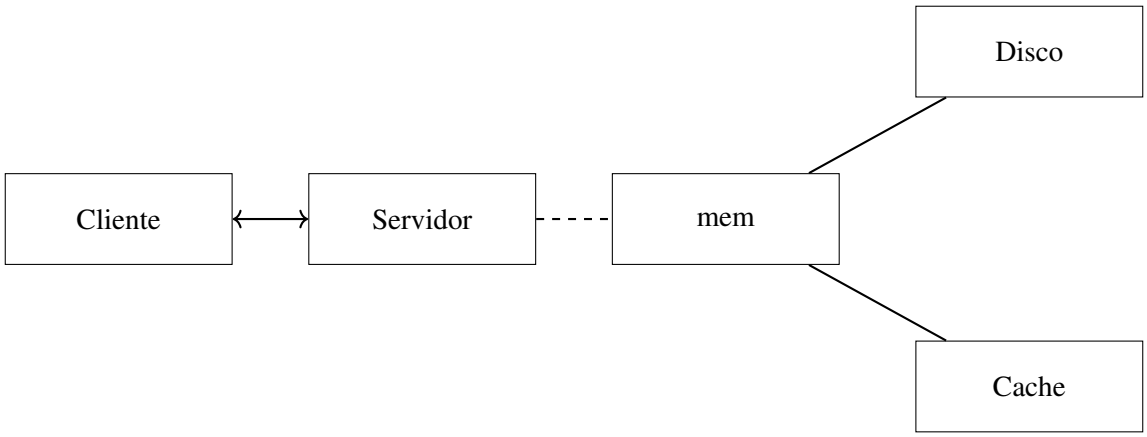
- **Comunicação entre processos** -garantir uma troca eficiente e segura de mensagens entre o cliente e o servidor
- **Concorrência no servidor** — assegurar que o servidor consegue tratar múltiplos pedidos de clientes em simultâneo, sem bloquear ou fazer com que um cliente
- **Gestão de memória** - gestão de forma eficiente a informação em memória(cache)

Assim neste relatório irei fazer uma breve análise sobre possíveis metodologias, uma explicação sobre a arquitetura implementada, uma explicação mais detalhada sobre as metodologias usadas nos pontos críticos identificados e uma análise de resultados.

Capítulo 2

Arquitetura

Inicialmente e para uma implementação mais fácil e de melhor gestão de erros dividimos o modulo do servidor em 2 partes. Servidor e mem. Sendo assim tudo que fosse relacionado com cache e gestão de memoria iria ser redirecionado para o modulo apelidade de "mem"sendo que o modulo servidor seria responsável por fazer apenas pedidos a cache e desse handle da melhor maneira a todos os clientes. Sendo esta possível representar da seguinte forma:



2.1 Possíveis metodologias

2.1.1 Primeira arquitetura

O desenvolvimento deste sistema sofreu uma brusca alteração já no final do mesmo sendo a grande diferença vista no módulo mem. Inicialmente foi usada uma metodologia de gestão de memória da seguinte forma: Todos os meta dados de ficheiros que não fossem possíveis ser guardados em memória por diversos fatores iriam ser então guardados num .txt ao qual apelidados de disco.txt.

Sendo assim sempre que fosse necessário uma consulta, remoção ou algum tipo de pesquisa se o mesmo não tivesse em cache nessa dada altura iria ser aberto esse disco.txt e iria ser feito a leitura gradual do documento usando as chamadas ao sistema (*open, read, lseek, close*). Desta forma garantimos que era feita a leitura total do ficheiro, que iria ser uma leitura linha a linha(não necessariamente, mas com o uso do lseek garantíamos que nenhuma linha iria ser passada a frente pois conseguimos ajustar o ponteiro de leitura para o final da linha anterior). Desta forma e se encontrado iria ser então feita uma gestão em que iríamos adicionar essa entrada a cache e passar uma entrada da cache para esse dado ficheiro.

Já em fase de análise de resultados não foi observado nada de anómalo a metodologia mas quando o servidor foi colocado sobre grandes cargas foi possível detetar que o custo de não ter essa meta informação em cache era muito grande pois desta forma havia um ficheiro muito grande em que estavam sempre a ser feitas chamadas ao sistema de *read* e *lseek* sendo que em grade escala esta metodologia provocava um overhead considerável, impactando negativamente a performance. Sendo que assim era necessário fazer uma reestruturação desse mesmo modulo. Desta forma surgiu então uma metodologia bastante mais efeciente a fazer essa gestão.

2.1.2 Segunda arquitetura

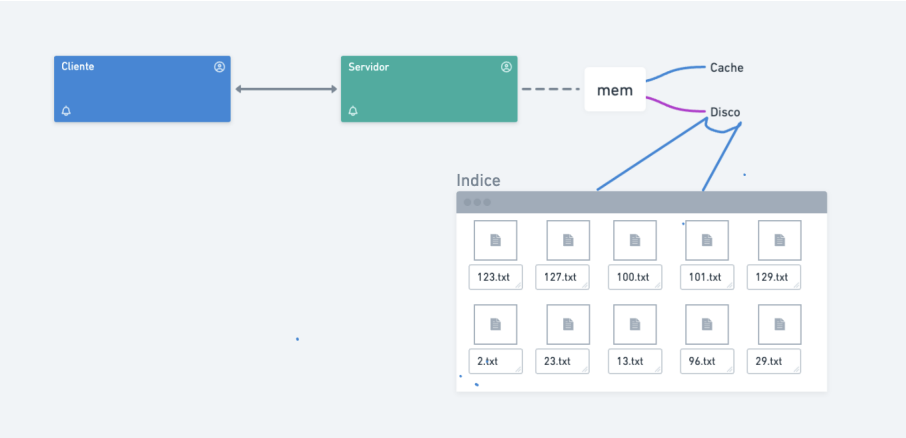


Figura 2.1: Imagem representativa da arquitetura

Assim sendo, alteramos a arquitetura do nosso sistema para a a arquitetura representada na imagem acima. Explicando esta um pouco melhor, começamos por criar uma pasta ao qual chamei de indice. Nesta, invés de guardar um único txt com todos os ficheiros que o nosso sistema tem, irei passar a guardar um txt em que dentro do mesmo apenas está toda a meta informação de um dado ficheiro sendo que o nome do mesmo iria ser composto pela sua key de inserção (ex: 29.txt, sendo 29 a key do ficheiro).

Desta forma irei conseguir poupar múltiplas chamadas ao sistema sendo que nesta arquitetura, uma simples consulta de ficheiro(operação recorrente pois existe 2 funcionalidades que envolvem consulta) iria ser feita com apenas uma função *access* e se o mesmo existi-se ira ser feito apenas um *read* pois toda a meta informação consegue ser realizada num unico *read*. A remoção do mesmo era feita pela remoção da cache se o mesmo estivesse em cache e após essa remoção apenas teria de ser feito um *execlp rm* sendo o custo deste também bastante reduzido comparado a nossa primeira arquitetura. Desta forma também conseguimos garantir quase "de graça" a persistência de dados pois a partir do momento que um documento fosse indexado, após a inserção do mesmo na cache iria ser criado o token do mesmo garantindo assim a persistência dos dados do nosso sistema.

Desta forma, conseguimos otimizar ao máximo as chamadas ao sistema, reduzindo o tempo de resposta para cada cliente. Isso permite escalar o sistema de maneira mais eficiente, possibilitando a realização de um grande número de operações sem comprometer a performance.

2.2 Comunicação entre Cliente e Servidor

Uma das principais necessidades do nosso sistema era haver uma rápida e segura comunicação entre os diversos clientes e o servidor. Desta forma levantei 2 principais requisitos:

- Servidor responder a clientes FIFO (primeiro cliente que "chegar"irá ser o primeiro cliente a ser atendido
- O servidor deve responder a cada cliente de forma precisa e acertada, isto é, o cliente deve receber apenas a resposta ao seu pedido

Assim escolhi o método de pipes com nome para responder a este requisito. Mas aqui é preciso ter em atenção um pequeno pormenor. Se o servidor responder a todos os clientes pelo mesmo pipe, a mensagem vai ser lida de maneira concorrente isto é, a resposta dada pelo servidor vai apenas ser lida pelo primeiro cliente(não necessariamente o primeiro cliente a fazer o pedido mas sim o primeiro cliente

que conseguiu ler a resposta). E como os pipes são FIFO, a mensagem apenas iria ser lida por um cliente podendo assim a mesma nunca chegar ao seu verdadeiro destino. Desta maneira irá ser preciso desenhar uma metodologia estável e que consiga responder a cada cliente de forma individual.

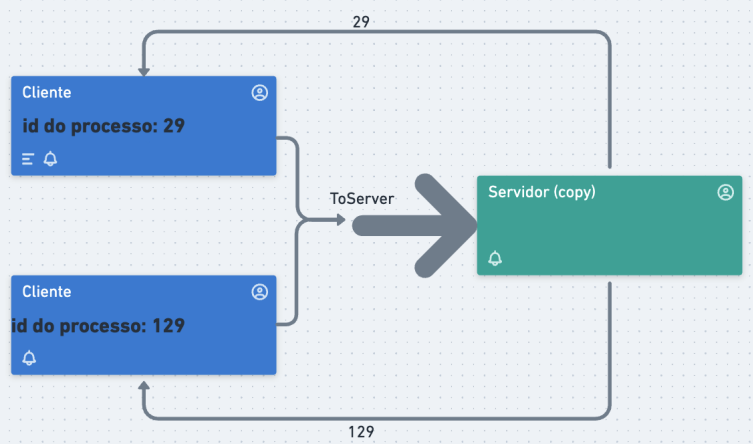


Figura 2.2: Imagem representativa dos Pipes entre Cliente e servidor

Desta forma, foi implementado o sistema representado pela imagem anterior. Este meio de comunicação iria ser implementado da seguinte forma: todos os pedidos realizados pelos clientes iriam ser enviados através do pipe "ToSever" . Desta forma o servidor iria receber e tratar os pedidos todos de forma FIFO, respondendo a todos os clientes consoante a sua ordem de chegada dos pedidos. Para garantir que a reposta e dada a cada cliente de forma única, cada pedido que fosse envidado para o servidor iria sofrer uma transformação antes do envio de maneira a que cada cliente crie um pipe pessoal de resposta e envie a informação do mesmo através do pedido. Foi escolhido para a nomenclatura do pipe o número de processo desse dado cliente. Assim desta forma conseguimos garantir que cada cliente irá criar um pipe único para receber a resposta ao pedido pois o numero de processo e único. Dessa forma o pedido iria seguir para o servidor com a seguinte estrutura "pipe(número de processo) pedido". Assim o servidor apos processar o pedido desse cliente iria responder ao pedido do mesmo para o pipe enviado pelo pedido garantindo assim que cada cliente recebe apenas a reposta do seu pedido.

2.3 Modulo Cliente

Sendo que a questão da comunicação entre servidor e cliente está resolvida podemos então partir para o resto do modulo do cliente. O modulo do cliente apenas terá então de:

- formatar o pedido para enviar para o servidor
- enviar o Pedido atraves do pipe *ToServer*
- Criar e esperar resposta no pipe pessoal
- ler e imprimir para o *std out* a resposta

2.3.1 Formatação do Pedido

Quanto a formatação do pedido, o cliente apenas irá juntar o seu id do processo, usando a função *getpid* ao inicio do pedido seguido seguido de todos os argumentos(*argv[]*) utilizando funções da biblioteca string.h como por exemplo *sprintf* e utilizando também *strlen* + 1 para enviar para sempre a mensagem na totalidade incluindo o '\0' de maneira a garantir que apenas o verdadeiro conteúdo do pedido é enviado para o servidor evitando "lixo"da memória. É feita também uma verificação do numero de argumentos para garantir que para o pedido efetuado, este contem toda a informação necessária para efetuar esse mesmo pedido.

2.3.2 Criação e uso de pipes no módulo Cliente

Como referido anteriormente, cada cliente irá sempre ter 2 pipes sendo um deles o pipe geral de pedidos *ToServer* responsável pelas mensagens cliente->Servidor e um pipe pessoal único responsável pelas mensagens Servidor->Cliente. Apenas o pipe pessoal é criado pelo cliente sendo que para isso usei a função *mkfifo* onde também assegurei que se a mesma não conseguisse criar a mesma devolve um erro (*perror*) e retornava logo 1.

Em caso de sucesso, poderíamos proceder então a abertura e escrita nos pipes usando funções: *open* e *write* tendo sempre em atenção o tamanho do pedido e o valor do retorno do *write*. Após isso e não menos importante asseguramos que fechamos o pipe de escrita para controlar o numero total de descritores abertos.

Por fim poderíamos então abrir o pipe pessoal e esperar a resposta do cliente. O mesmo fica num ciclo while a ler o seu pipe pessoal até chegar o EOF (momento que o servidor fecha pipe do cliente). Após a leitura ser efetuada e com atenção ao valor de retorno no numero de total de bytes lidos podemos então imprimir a mensagem para o *std out* do cliente.

Também foi tido em conta ter sido feito o unlink do pipe pessoal para remover corretamente o FIFO do sistema de ficheiros após a sua utilização.

2.4 Modulo Mem

Este modulo e responsável por toda a gestão dos dados em disco, desde adicionar, remover, gerar um DocInfo (estrutura utilizada para armazenar e manipular os meta dados de cada ficheiro) e gerar a partir dos dados atuais do sistema, conjuntos predefinidos de dados usados posteriormente pelo modulo servidor.

2.4.1 DocInfo e chavePath

Inicialmente comecei por definir uma estrutura ao qual apelidei de DocInfo em que esta é composta por: *int id, char titulo[200], char autor[200], char ano[5], char path[64]*. O int id é referente a key do dado ficheiro, e seguido do seu titulo, autores, ano e path. Esta estrutura simples permite fazer toda a gestão consulta e a funcionalidade de pesquisa -l.

Sendo que para cumprir a funcionalidade -s, e devido a usar todos os documentos indexados no sistema até esse comando, utilizamos uma versão mais reduzida de DocInfo ao qual apelidei de chavePath que apenas é composta por: *int chave, char path[64]* pois estas são as únicas informações necessárias e assim conseguíamos uma melhor gestão de memoria.

2.4.2 Organização de DocInfo e ChavePath

Agora com ambas as estruturas base criadas podíamos passar agora para uma maneira de organizar toda estas estruturas de maneira a promover a eficiência e escalabilidade do sistema. Desta maneira foram então desenhadas duas estruturas diferentes:

- Hashtable
- Lista Ligada

A hashtable ira ser o elemento central da nossa estrutura. Esta irá ser responsável guardar todos os docInfo pedidos dada uma key. Desta maneira garantimos rapidez e escalabilidade pois como todas as operações são efetuadas dada uma key conseguimos então com um custo médio de O(1) consultar essa

key, inserir, e remover. Assim apenas nos resta implementar uma maneira de gerir quais keys remover e até mesmo saber o numero total de dados guardados, assim como o espaço que a mesma ainda tinha livre.
Desta forma optamos então pela implementação de uma nova estrutura,cacheTipo, composta por: *int total, int usados, GQueue* cache*. Desta forma conseguimos então definir um tamanho total, saber rapidamente quantos DocInfo foram guardados na hash e a lista ligada servia para guardar a ordem de acessos/adições de cada DocInfo sendo apenas uma lista ligada das várias keys em cache facilitando posteriormente a implementação de políticas de cache.

2.4.3 Funcionalidades Principais

Como referido anteriormente as funcionalidades deste módulo passam por: inserção, gerar DocInfo para pedido de consulta, remoção da meta informação guardada, e gestão da cache.

Inserção

Quanto a inserção de uma dada meta informação geramos um DocInfo recorrendo a funções *strcpy*. Se a cache ainda tiver espaço livre, isto é, feito o calculo da diferença dos dois valores em *cacheTipo*, adicionamos essa DocInfo a hastable com uma key gerada. Se a mesma não tiver espaço, iremos chamar uma função ao qual apelidei de *gestaoCache* que posteriormente neste relatório, irei abordar melhor, e a seguir voltamos a tentar adicionar adicionar o DocInfo (que ira ser com sucesso pois gestaoCache irá remover da cache um DocInfo consoante a política de gestão de cache). Por fim era gerado token(documento .txt) apelidado com a key do DocInfo, com todas as informações necessárias escritas no mesmo. Desta forma conseguimos garantir a persistência dos dados mesmo se a execução do servidor der erro. Já em fase de testes e analises de tempo e gestão de recursos foi pensado em dividir o trabalho de inserção em 2. Processo pai iria ter todo o trabalho inicialmente e quando apenas resta-se a criação do token ele ia criar um filho para tratar desse trabalho ficando assim o mesmo pronto para responder a outro pedido.

Consulta

Já relacionado à consulta, foi pensado uma versão mais generalizada de função para a mesma poder ser aplicada em outros contextos também. Assim a função chamada *getDoc* é uma função que recebe uma key como argumento, e inicialmente vê se existe algum DocInfo na hash com essa key usando uma função da glib. Se a mesma a mesma retornar NULL então sabemos que temos de ir ao disco. Então e de maneira eficiente, tentamos primeiro ver se existe algum documento com essa key. Como os ficheiros estão nomeados com a sua key, apenas é necessário o uso da função *access*. Se o mesmo existir abrimos o txt e preenchemos todos os campos do DocInfo e retornamos o DocInfo em caso de existir ou NULL caso contrário.

Remoção

Já em relação a remoção de uma dada key e de maneira a ser eficiente, dividimos a mesma em 3 funções. Utilizando de novo a hash pois esta é a que oferece melhor eficiência, verificamos se a dada key se encontra atualmente em cache. Se ele tiver em cache, apagamos da cache e chamamos a função responsável pela remoção da lista ligada. Caso contrario partimos logo para a remoção dessa key do disco. Caso esta não se encontre na hashtable sabemos que também não estará na lista ligada então procedemos logo para a remoção do disco. Na remoção da lista é tido em conta que para alem de remover da lista é necessário decrementar o valor do numero total de DocInfo em cache e de seguida é chamada a remoção do disco. Já na remoção do disco inicialmente foi optei por criar um filho e o mesmo fazer *execlp("rm", "rm", path, NULL).*

Mais tarde e depois já de uma análise de resultados provei que esta maneira se provava ineficiente pois todo o processo de criar um filho, filho executar o comando e esperar pela morte do filho era bem mais trabalhosa que a simples função *unlink(path)*. Sendo o uso de unlink mais fácil e rápido foi a maneira escolhida para a solução deste problema.

Talvez inserir imagem

Gestão da Cache

Depois uma das principais funções deste modulo é a gestão da cache. Esta vai ser essencial para manter uma ordem na nossa cache e saber sempre qual item remover consoante a política. De maneira a facilitar a troca de políticas de cache o código esta implementado de maneira a que basta trocar uma variável global do *cache.h* chamada de *TIPOCACHE*. Esta gestão de cache e dividia em 2 grandes funções com propósitos bem diferentes. A primeira função apelidei de *gestaoConsulta()*. Esta é responsável por gerir a ordem correta da lista ligada. Recebe como argumento uma key (key usada para adicionar/consultar/procura(-l)) e se este elemento esta em cache(consulta feita na hash) nessa dada altura irá remover essa key da lista e adiciona-la no final. Assim desta forma conseguimos garantir que a lista ligada esta sempre ordenada do DocInfo usado mais antigo até ao mais recente. Se o mesmo não esta em cache chama a função *gestaoCache()* que irei explicar a seguir, que remove um DocInfo consoante a política, e depois disso adiciona a key dada na ultima posição da lista ligada e adiciona o DocInfo na hashtable.
A segunda função grande função apelidei de *gestaoCache()*. Esta função é responsável por escolher qual DocInfo remover da cache. E aqui é quando realmente damos uso à lista ligada. Sendo assim a função funciona da seguinte forma:

- Se CACHETIPO = 0 (LRU) remove a cabeça da lista ligada(elemento mais antigo)
- Se CACHETIPO = 1 (MRU) remove o ultimo elemento (elemento usado mais recentemente)
- Se CACHETIPO = 0 (Random) remove um DocInfo escolhido aleatoriamente

Desta maneira e com o uso destas duas funções conseguimos garantir uma gestão eficiente e fidedigna ao tipo de cache utilizada e conseguimos garantir que a cache com o uso destas funções sempre que necessário que o limite de cache é respeitado, seguindo sempre um padrão válido para o tipo de cache em uso.

Estrutura em uso na query -s

Por fim também usamos uma estrutura bastante mais simples para a query -s. Sempre que necessário fazer esta query, a cache organiza uma listaS que é uma lista de uma estrutura composta apenas pelo path e key. Para preenchermos listaS utilizamos funções *opendir* e *readdir* para percorrer toda a diretoria dos dados em disco e preenchemos assim toda essa listaS utilizando *getDoc()*. Escolhi usar este método de listaS pois permite ao servidor dividir o trabalho da melhor maneira entre os vários processos sem o uso de concorrência entre processos.
Sendo este um modulo com bastantes chamadas open, foi tido o problema de ter vários descritores abertos então fiz uma analise total a todas as funções e especialmente depois de perror's para garantir que todos os descritores eram fechados devidamente.

2.5 Modulo Servidor

Este modulo é o responsável por receber todos os pedidos, processamento e envio respostas e chamar o modulo mem sempre que necessário (ligados através do compilador e utilização de include). Sendo

assim este modulo ira ter as seguintes características:

- Criar Pipe responsável pela comunicação entre cliente->servidor e ler todos os pedidos efetuados através do mesmo
- Processar os pedidos
- responder para o cliente

Sendo este um modulo que irá criar filhos vai ser necessário ter bastante cuidado com a a criação dos mesmos e garantir o devido tratamento de processos zombie e como também ira ter chamadas de *open* vai ser preciso garantir que o mesmo também irá fechar os descritores sempre que possível.

2.5.1 Criação do Pipe Cliente->Servidor e leitura do mesmo

Como referido anteriormente é necessário criar um pipe para o cliente poder comunicar com o servidor. Ao usar este tipo de ferramenta conseguimos garantir por si só que todos os clientes vão ser atendidos por ordem de chegada. Então o processo servidor vai ser responsável da criação do pipe *ToServer* através da função *mkfifo* e o mesmo, antes de terminar a sua execução(através de um pedido -f de um cliente) ira fazer um *close* do seu pipe e posteriormente unlink para garantir a correta remoção do ficheiro especial criado para a comunicação.
Durante a execução o mesmo irá abrir o sue pipe e ficar constantemente a fazer *read* do mesmo a espera de um pedido de cliente. Assim que receba um pedido de um cliente irá tratar desse pedido.

2.5.2 Processar Pedidos

Todos os pedidos efetuados para o servidor estão padronizados. O padrão segue as seguintes normas *"Int(referente ao ID do cliente) pedido"*. Assim desta forma e com recurso a biblioteca *string.h* iremos partir o pedido em vários tokens divididos por espaços que irão ser referentes a todas as necessidades de cada pedido em especial. Em relação a este tema foram divididos os pedidos em 2 grandes categorias, os pedidos que podiam ser feitos de forma concorrente(consulta e operações de procura) e os pedidos que tinham de ser realizados de forma sequencial (adição,remoção e shutdown) sendo que deste modo o servidor apenas pode bloquear enquanto estiver a adicionar ou a remover.

Adicionar,remover e consulta

Com a modularidade aplicada neste projeto, o pedido de adição e remoção, depois de "partir"o pedido pelos tokens necessários. Após termos todos os *char** necessários para a adição e remoção, basta chamar as funções *insereCache(key,titulo,autor,ano,path)* e *apagaCache(key)* respetivamente sendo que estas retornam exatamente o que o cliente quer como reposta. A adição ira retornar a key que o DocInfo foi inserido e a remoção ira retornar 1 em caso de sucesso e 0 caso contrário. Já quando o pedido for uma consulta, o processamento vai seguir o padrão anterior sendo que este pedido ira chamar a função *getDoc(key)*. Este ira receber *NULL* em caso de não existir nenhum documento indexado e formata uam reposta adequada para o cliente com essa informação. Em caso de sucesso formata uma string com toda a informação guardada em DocInfo. Tem apenas uma particularidade essencial para garantir a correta gestão de cache. Sempre que recebe um DocInfo válido, a mesma chama *gestaoConsulta(key)* que, como referido anteriormente para manter uma correta gestão da cache.

Operações de Procura

Quanto as operações de procura temos 2 tipos de procura nos ficheiros indexados. Temos a opção -l que dada uma key e uma palavra deve responder o numero total de linhas que e possível encontrar a mesma palavra e a opção -s que procura em todos os ficheiros indexados por uma dada palavra devolvendo uma lista de keys referentes a todos os documentos que contem essa dada palavra.

Operação -l

Para o pedido relacionado a -l optei por usar a criação de um filho para executar um comando bastante conhecido *grep -c path* que devolve exatamente o valor pedido. Era possível o uso de *strstr* mas foi optado o método d grep para por em pratica os conceitos praticados nas aulas práticas da UC. Deste modo para o pedido -c chamo a minha função *contaLinhas(palavra,key,resposta*. Como irei recorrer ao comando *execlp* irá ser necessário a criação de um processo filho para o mesmo o fazer visto que após esta função o processo ira terminar. Posto isto era necessário criar uma maneira de o processo filho comunicar o resultado com o processo pai antes de terminar. Para isso criei um pipe anónimo. Após a criação do mesmo, recorri a outra função *dup2* para direcionar o output do *std out* para o pipe anónimo. O processo pai irá então ler do pipe a resposta e escrever para o pipe dado na chamada de função (resposta). Por fim para garantir o controlo de processos zombie o pai faz um *waitpid(pid, &status, 0)*. Como a função faz bastantes open´s foi revista para garantir que nenhum descritor ficava por fechar e se todos era fechados quando fosse possível para o processo pai não ficar bloqueado a espera de um close do pipe anónimo. Como foi um pedido relacionado a um único documento achei pertinente voltar a fazer uma *gestaoConsulta(key)* para garantir uma boa gestão da cache.

Operação -s

Este pedido é feito de uma forma bastante especial. O cliente faz um pedido de procura de uma palavra por todos os documentos indexados e define um numero de processos que o sistema pode criar para responder a este pedido.

Como o modulo mem já gera uma listaS organizada com todos os documentos indexados e suas keys, esta função apenas é necessário pedir essa ao modulo essa lista para ter toda a informação necessária para resolver o pedido. Como esta função retorna o numero de ficheiros nessa dada lista a divisão entre processos segue uma lógica de divisão inteira garantimos que todos os processos irão receber a mesma carga sendo que o ultimo poderá fazer n-1 entradas. Como envolvia processos filhos iria ser preciso seguir a mesma lógica do pedido -l e criar pipes anónimos para a comunicação entre o processo filho e o processo pai.

Assim e com a toda a lógica já pensada apenas era preciso implementar. Então temos um ciclo que cria todos os filhos pedidos e estes calculam o seu limite inferior e superior. Posteriormente abrem cada ficheiro, fazem *read* e processam o buffer que leram usando *strstr*. Em caso de sucesso paravam o *read* fechavam o ficheiro e escrevem o id desse path para o pipe anónimo. No final fecham o pipe e dão *exit*. O pai apenas fica a ler ate ao fim do pipe(EOF) as respostas, faz um *wait* pelos filhos todos(que em situações normais já todos os filhos acabaram pois já todos fecharam o pipe de escrita), e por fim processa a string para responder ao cliente num formato adequando. Desta forma conseguimos garantir que não permitíamos a existência de processos zombie do lado do servidor.

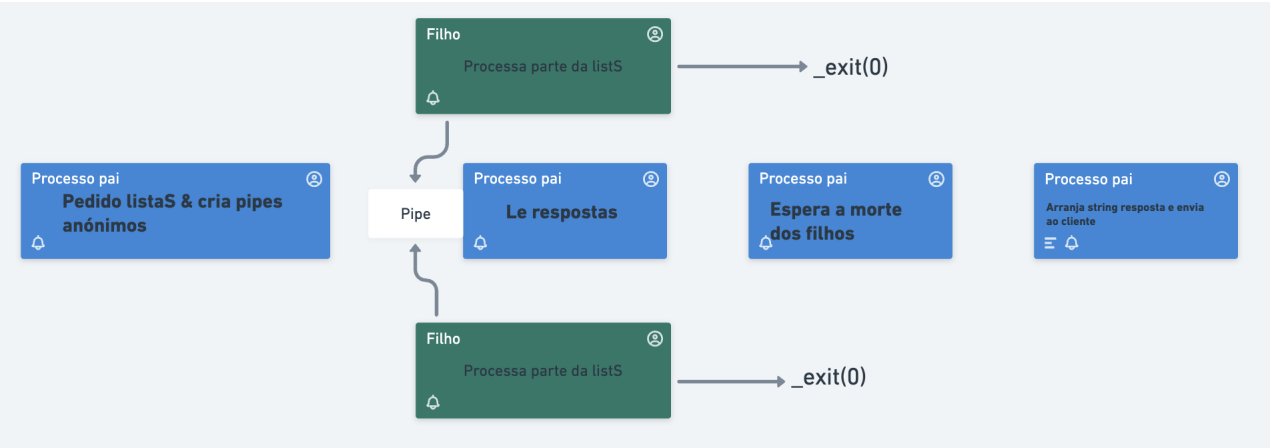


Figura 2.3: Fluxo da função `palavraChave`: paralelismo com filhos, pipes e um resumo abreviado das funções de cada processo em cada estágio

Servidor não bloqueante

Como referido anteriormente, um dos objetivos que tracei para este projeto era que o servidor ficasse o menor tempo possível bloqueado. Desta forma apenas achei pertinente o mesmo ficar bloqueado apenas a adicionar/remover DocInfo's e quando recebe um pedido de desligar. Desta forma no momento após a leitura, é feita uma breve análise do tipo de pedido. Se o pedido for do tipo adicionar remover ou desligar processa o próprio servidor. Caso contrário, o servidor cria um filho para processar e responder ao pedido e termina após isso, podendo assim o servidor (processo pai) receber e tratar mais um pedido e assim sucessivamente até receber um pedido de desligar sem a necessidade de ficar bloqueado quando não seja necessário.

Gestão processos zombie

Com o tipo de abordagem escolhido surgiu então um novo desafio, a gestão dos processos zombie. Como o processo pai(servidor) prosseguia logo para atender outro pedido e nunca consultava o estado dos filhos que criava, estes ficavam em estado zombie no sistema podendo assim correr o risco de esgotar todas as entradas da *process table* pois esta é de tamanho finito, ficando assim impedidos de criar novos processos e esgotando todo o espaço do *kernel* com informações relativas aos pid, pid do pai, código de saída entre outros. Desta forma foi pensado em 2 soluções possíveis, ou uso de sinais ou informar o pai que o filho terminou de alguma maneira. Foi optado a segunda opção pois quase tudo que era necessário para a implementação já estava desenvolvido.

Como já existia uma maneira de comunicar com o servidor, usando o pipe com nome *ToServer* e este recebendo pedidos do género "pid pedido"usamos este processo para implementar. Desta forma antes de cada filho executar o seu _exit ele envia então o seu pid seguido de -k para o pipe em que o servidor recebe os pedidos. Sendo que então apenas era preciso fazer o servidor ao perceber que recebia um pedido -k bloquear, para o mesmo fazer waitpid do pid do pedido. Desta forma conseguimos garantir que evitávamos a existência de múltiplos zombies no sistema.

Como estava a adicionar mais um tipo pedido que podia bloquear o servidor, pois tinha de ser o próprio a fazer waitpid, foi pensado em usar uma espera não bloqueante usando a flag *WNOHANG*. E usava esse valor para determinar a próxima operação. Se fosse um erro fazia o devido tratamento, se fosse 0 voltava a mandar um pedido para o pipe *ToServer* com o pid que tinha esperado para o pedido voltar para o final da stack de pedidos, e ser processado mais tarde.

Em fase de teste foi observado que embora os processos filhos tivessem a enviar o pedido ao pai

na operação antes do seu *exit* e o mesmo ainda tivesse de ler e processar, o pai chegava muitas vezes primeiro ao *waitpid*. Sendo assim foi escolhido usar um *waitpid* bloqueante usando a flag 0 pois o custo da espera era insignificante comparando com o total de zombies possíveis, e até mesmo o tempo de voltar a processar um novo pedido.

Funções extra adicionais

Gestão de duplicados

Um dos grandes problemas atuais na gestão de cache e de disco passa pela identificação e gestão de dados duplicados. Embora o enunciado não abordasse esse tema foi implementado uma função que permitia o controlo e gestão de ficheiros duplicados. Dessa maneira conseguimos ter um controlo e gestão de espaço em disco e cache muito maior. Também irão ser feitos testes relativos ao peso dessa mesma inserção. Esta função define como ficheiro duplicado aquele que tem todos os campos iguais a um ficheiro já indexado.

Método de gerar keys

Embora, para melhorar o controlo de inputs e outputs nos testes, tenha sido adotada uma estratégia de geração incremental de keys (onde cada nova key corresponde à anterior + 1), foi também desenvolvida uma função alternativa para gerar chaves aleatórias dentro de certos critérios. Esta abordagem visa introduzir uma camada adicional de segurança, dificultando a previsibilidade dos identificadores utilizados.

Cuidados com a memoria

Como abordado nas aulas práticas e teóricas, a criação de um filho implica a duplicação do espaço de memória do processo pai, incluindo variáveis, descritores e outros, mas quando posteriormente fizer alguma alteração, apenas irá alterar para o filho sendo que o pai não tem acesso a essa informação.
Desta forma, adicionamos mais um comando ao servidor, o "*assinatura* -u *key*". A assinatura, e o id do processo que gera o pedido -u para assim haver controlo em logs posteriores, e a key representa a key que o filho acedeu. Desta forma sempre que o filho faz uma consulta envia uma mensagem para o *ToServer* para o pai fazer uma gestão da cache dele, assim garantimos a coerência da cache.

Capítulo 3

Resultados e Analise

Nest capitulo irei testar todos os tipos de cache com diversos tipos de carga e fazer uma análise de resultados obtidos. Estes testes irão ser efetuados num MacBook pro m4 tendo com processador um Apple M4 Pro um disco ssd e uma mória ram de 24GB.

3.0.1 Testes iniciais a cada tipo de cache

Sabemos que avaliar uma cache quanto a sua performance pode ser sempre discutível, pois dependendo do tipo de carga, a mesma cache tanto pode oferecer os melhores resultados como num outro teste ser a pior. Então de uma maneira inicial iremos correr uma script com inputs pre determinados e calcular os tempos médios de cada operação, sendo o numero total de 1015 operações(apenas teste 2 têm x entradas) com os primeiros 15 comandos de inserção -a para povoar a cache inteira. Desta forma o server e corrido com a identificação que o numero total de DocInfo´s em cache sendo 15. Para aumentar a qualidade do teste foram feitos 10 testes seguidos com o mesmo input e feito uma média dos 3 melhores resultados a nível de tempo global.

Tipo de Cache	Teste 1	Teste 2	Teste 3
LRU	8,47seg	0,7567seg	8,92seg
MRU	8,678seg	0,7398seg	8,9seg
Random	8,9seg	0,7553seg	8,68seg

Tabela 3.1: Resultados comparativos entre tipos de cache

Desta forma conseguimos concluir que não existe claramente um vencedor no que toca a tipo de organização de cache pois sempre vai depender do tipo de utilização, isto é, dependendo do tipo de carga e da ordem dos documentos acessados, uma cache tanto pode ser a que tem o melhor desempenho como ao mesmo tempo, ter o pior desempenho.

- Um padrão de acesso a ficheiros repetidos e recentemente adicionados/consultados, favorece uma cache do tipo LRU
- Um padrão de acesso a ficheiros antigos e adicionados a mais tempo ou menos consultados, favorece uma cache do tipo MRU
- Um padrão de acesso random ficheiros tende a favorecer uma cache do tipo random

Mesmo que tentamos calcular a media dos 3 resultados(anexo exel) conseguimos deparar que o mais rápido embora seja a cache random, esta não consegue manter um tempo constante logo acaba por ser imprevisível, isto é , tanto pode ser a melhor durante a 1h de pedidos como a pior. Já em relação a MRU e LRU, como os tempos foram medidos com um ssd o custo de acesso a disco não foi tão pesado fazendo com que apenas houvesse uma diferença de 1%. Sendo o custo de operações de ambas as gestões iguais, o único fator iria ser mesmo a qualidade do disco que no caso do teste, não teve grande impacto, presumindo assim que um teste realizado num disco HDD iria mostrar ganhos maiores.

3.0.2 Testes relativos a duplicação

O numero total de operações para os testes do sistema foi baseada em estudos reais sobre o comportamento de sistemas de ficheiros. Em particular, utilizei como referência o estudo"A Five-Year Study of File-System Metadata"Agrawal et al. (2007).

Este estudo analisou padrões de acesso a sistemas de ficheiros de mais de 60.000 máquinas ao longo de cinco anos, concluindo que operações de leitura são substancialmente mais frequentes do que operações de escrita.

Com base nestes dados, e considerando o contexto específico do nosso serviço de indexação e pesquisa de documentos, foi adotada a seguinte distribuição aproximada para a geração de testes, sendo o numero total 11100 operações. Assim desta forma conseguimos aumentar a carga e pressão submetida ao servidor usando aproximadamente cerca de 40% a 50% de inserções repetidasZhu and Strunk (2016). Iremos também tentar calcular o tamanho total de informação em disco já que esta poderá ser uma métrica boa para compensar o tempo de inserção maior.

Operação	**Descrição**	**Frequência Estimada**
`-a`	Indexação de novos documentos	14%
`-c`	Consulta de metainformação	27%
`-d`	Remoção de metainformação	5%
`-l`	Contagem de linhas com palavra-chave	27%
`-s`	Pesquisa de documentos por palavra-chave	27%

Tabela 3.2: Distribuição de operações usada para a geração de testes.

Esta distribuição pretende refletir um cenário realista, onde as operações de leitura e pesquisa dominam a utilização do sistema, tal como observado nos padrões analisados no estudo referido apenas aumentado as inserções para aumentar o numero total de ficheiros no sistema e tentar aproximar mais de um sistema já com alguma povoação. Neste teste iremos testar o impacto da duplicação e gestão de ficheiros duplicados. Como este teste visa apenas o impacto da duplicação, apenas foi usado uma cache do tipo LRU. Os resultados obtidos são os seguintes:

LRU	**Teste 1 c/dups**	**Teste 2 s/dups**	**Ganhos/Perdas (%)**
Tempo total	191,41	166,09	13,228149
$ -a	0,0058	0,0083	-43,10344828
$ -d	0,0056	0,0055	1,785714286
$ -c	0,006	0,0059	1,666666667
$ -l	0,006	0,0059	1,666666667
$ -s	0,037	0,026	29,72972973
Espaço Disco (kb)	6484	3284	49,3522517

Tabela 3.3: Comparação de desempenho LRU entre testes com e sem duplicações.

Com estes resultados podemos concluir que para uma melhor otimização e considerando um volume maior de clientes, a consideração pelos ficheiros duplicados tem um grande impacto na eficiência de espaço e tempo geral. Assim é possível concluir que a duplicação tem um grande impacto na gestão de espaço e tempo de resposta. Embora a inserção sofra uma penalização a mesma é compensada em operações de consulta e procura. Também e possível observar que a nível de gestão de espaço utilizado a mesma se revela bastante importante pois permite um melhor aproveitamento de espaço promovendo consequentemente uma melhor gestão de ficheiros em cache pois estes serão todos únicos.

Capítulo 4

Conclusão

Este estudo incidiu sobre o desenvolvimento de um sistema de gestão de ficheiros de raiz usando vários tipos de métodos a nível de gestão de cache e comunicação entre processos.

Apesar de haver um grande nível de satisfação com o projeto final, é possível apontar melhorias possíveis e mais otimizações ao mesmo. Estas não foram possíveis pois havia algum nível elevado de não determinismo no sistema que foi considerado prioridade em relação ao resto. Embora grande parte fosse resolvido a sensação que a qualquer altura poderia acontecer era grande embora apenas fosse possível observar em testes de larga escala.

Já a nível de testes não fosse possível obter conclusões óbvias, o impacto do tipo de pedidos e de acessos a ficheiros a nível de eficiência de gestão de cache foi visível mostrando que não existe uma solução ótima. Qualquer solução irá ter os seus pontos positivos e negativos, sendo que um balanceamento entre vários tipos de gestão possa ser a melhor opção.

Embora não fosse um requisito, a duplicação revelou se de facto como um dos pontos chaves no desempenho superando qualquer ganho relacionado ao tipo de gestão de cache. Assim , e com base no meu sistema e nos testes realizados concluo que tendo em conta os duplicados e aplicando um método misto de cache, seria a melhor opção.

Uma das maiores revelações do desenvolvimento deste sistema foi uma gestão aleatória. Embora esta teoricamente pareça ser uma má gestão, esta provou ser competitiva com sistemas de gestão mais tradicionais. Uma das possíveis justificações e o facto de o teste ser gerado aleatoriamente pode beneficiar de alguma forma este tipo de gestão, por vezes tornou se difícil arranjar um teste "mau"deste tipo de gestão.

Foi considerado o uso de uma árvore binária balanceada em que nesta ficava organizado a frequência de uso de cada ficheiro de maneira a implementar um sistema ainda mais justo de substituição. O objetivo da mesma era completar uma gestão random de maneira a fazer um random n entre os n ficheiros menos utilizados de maneira a fazer um sistema random mais preponderado.

Bibliografia

Agrawal, N., Bolosky, W. J., Douceur, J. R., and Lorch, J. R. (2007). A five-year study of file-system metadata. In *Proceedings of the 5th USENIX Conference on File and Storage Technologies (FAST '07)*. USENIX Association.

Zhu, Y. and Strunk, J. D. (2016). A long-term user-centric analysis of deduplication patterns. In *2016 IEEE 32nd International Conference on Massive Storage Systems and Technology (MSST)*, pages 1–10. IEEE.